

Correção de Rotina de Web Scraping em Portal de Notícias: diagnóstico, solução e avaliação de qualidade de dados

Hellen de Andrade Moura

Processo Seletivo — NetLab, Laboratório de Estudos de Internet e Redes, UFRJ

Resumo

Este documento apresenta o diagnóstico, a solução técnica e a avaliação de qualidade de dados de uma rotina de *web scraping* utilizada pelo NetLab UFRJ para coletar resultados de busca do portal G1 pelo termo “LGPD”. A investigação identificou que a falha da rotina original decorria de renderização *client-side*: o conteúdo dos resultados não estava presente no HTML estático entregue pelo servidor, mas era injetado via JavaScript. A solução migrou a extração para o consumo direto da API JSON interna do G1 (`busca.globo.com/v1/search`), mantendo o BeautifulSoup apenas para limpeza de marcação HTML residual. Foram implementados tratamento de erros por página, um *circuit breaker* para falhas consecutivas e deduplicação por identificador primário. A avaliação de qualidade dos dados, realizada sobre uma amostra de referência de 20 itens, atingiu 100% nas dimensões de atualidade, precisão, acurácia, unicidade, consistência e rastreabilidade, com 95% de completude no campo de resumo.

Palavras-chave: web scraping, engenharia de dados, qualidade de dados, API JSON, Python

Diagnóstico

A rotina original utilizava `requests` e `BeautifulSoup` para coletar resultados da URL `https://g1.globo.com/busca/?q=lgpd`, mas retornava zero ou poucos registros. A investigação revelou três camadas de problema: bugs de lógica no código, renderização *client-side* e paginação por *offset* (padrão Elasticsearch), e não por parâmetro `&page=N`.

Bugs de lógica no código original

- **Sobrescrita de resultados.** A atribuição `resultados = dados_pagina` dentro do loop substitui o acumulador a cada iteração; apenas a última página sobrevive.
- **AttributeError não tratado.** `card.find(...).get_text()` levanta exceção quando o seletor retorna `None`, interrompendo toda a execução.
- **Paginação incorreta.** `range(TOTAL_PAGINAS)` começa em 0 e não corresponde ao esquema real do site.
- **Sem verificação de status HTTP.** Nenhuma chamada a `raise_for_status()` antes de processar a resposta do servidor.

Renderização *client-side* — achado principal

Uma busca pelo termo “LGPD” no HTML bruto (*view-source*) retornou **0 ocorrências**. Na página renderizada pelo navegador (F12 → Elements), o mesmo termo aparecia **17 vezes**. Os resultados são, portanto, injetados via JavaScript após o carregamento inicial, nenhuma correção de seletor BeautifulSoup resolveria o problema. Um componente `allSearchResults`, carregado pelo script `backstage-cms-all-search-results`, confirmou a arquitetura *client-side*.

A API real identificada via DevTools

Via DevTools → Network → Fetch/XHR, foi identificada a chamada que a interface web realiza para popular os resultados:

- **URL:** `https://busca.globo.com/v1/search` (POST, não GET).
- **Payload:** array de objetos com `search_profile`, `query` e `params` (`q`, `from`, `size`). O perfil relevante é `sp_g1_globo_com` com `g1.info_query_recency`.
- **Paginação Elasticsearch:** parâmetros `from/size`; p.ex. página 2 $\equiv$ `from: 10, size: 10`.
- **Estrutura da resposta:** `resposta[0]["result"]["hits"]["hits"]` → lista de artigos, cada um com campo `_source`.

Problemas de qualidade nos dados brutos

- **URL de rastreamento.** O campo `_source.url` frequentemente contém um link `measures.globo.com/v1/click?...&u=url_real`. É necessário decodificar o parâmetro `u`.
- **Campo de resumo instável.** Artigos podem apresentar `caption`, `summaryBlocks` (formato Draft.js aninhado) ou apenas `highlight.body` com marcação HTML. Ordem de *fallback* adotada: `caption` → `summaryBlocks` → `highlight.body` limpo.
- **Metadados ausentes na API.** Número de página e horário de coleta são calculados pelo próprio script.

Solução — Código-Fonte

A extração principal foi migrada para o consumo direto da API JSON. O BeautifulSoup foi mantido exclusivamente para remoção de tags HTML no campo `highlight.body`. O projeto é organizado em quatro módulos: `src/scrapper_g1.py` (coleta e persistência), `tests/test_extracao.py` (testes unitários), `tests/test_stress.py` (testes de robustez) e `llm_recovery.py` (proposta de integração com LLM).

Configuração e montagem do *payload*

`src/scrapper_g1.py -- configuração`

```
1 HEADERS = {
2     "Content-Type": "application/json",
3     "Accept": "application/json, text/plain, */*",
4     "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) ...",
5     "Origin": "https://g1.globo.com",
6     "Referer": "https://g1.globo.com/",
7     "x-tenant-id": "g1",
8     "x-track-urls": "false"
9 }
10
11 def montar_payload(termo_busca, from_, size):
12     return [{
13         "search_profile": "sp_g1_globo_com",
14         "query": "g1.info_query_recency",
15         "params": {"q": termo_busca, "from": from_, "size": size}
16     }]
```

Requisição com tratamento de erros

buscar_artigos_com_total()

```
1 def buscar_artigos_com_total(termo_busca, from_, size):
2     url = "https://busca.globo.com/v1/search"
3     try:
4         r = requests.post(url, headers=HEADERS,
5                             json=montar_payload(termo_busca, from_, size))
6         r.raise_for_status()
7         dados = r.json()
8         hits = dados[0]["result"]["hits"]
9         return hits["hits"], hits["total"]["value"], True
10    except requests.exceptions.JSONDecodeError as e:
11        print(f"Erro JSON from={from_}: {e}"); return [], 0, False
12    except requests.exceptions.RequestException as e:
13        print(f"Erro HTTP from={from_}: {e}"); return [], 0, False
14    except (KeyError, IndexError) as e:
15        print(f"Estrutura inesperada from={from_}: {e}"); return [], 0, False
```

Extração de campos com *fallback*

extrair_dados_artigo()

```
1 def extrair_dados_artigo(artigo, pagina, coletado_em):
2     fonte = artigo.get("_source") or {}
3
4     url_bruta = fonte.get("url") or ""
5     if "measures.globo.com" in url_bruta:
6         params = urllib.parse.parse_qs(
7             urllib.parse.urlparse(url_bruta).query)
8         url_final = params.get("u", [url_bruta])[0]
9     else:
10        url_final = url_bruta
11
12    resumo = ""
13    if fonte.get("caption"):
14        resumo = str(fonte["caption"])
15    elif isinstance(fonte.get("summaryBlocks"), list):
16        textos = [
17            str(bloco["text"])
18            for item in fonte["summaryBlocks"]
19            if isinstance(item, dict)
20            for bloco in (item.get("blocks") or [])
21            if isinstance(bloco, dict) and bloco.get("text") is not None
22        ]
23        resumo = " ".join(textos)
24
25    highlight = artigo.get("highlight") or {}
26    if not resumo and isinstance(highlight.get("body"), list):
27        resumo = BeautifulSoup(
28            str(highlight["body"][0]), "html.parser"
29        ).get_text(strip=True)
30
31    return {
32        "titulo": fonte.get("title") or "",
33        "resumo": resumo.strip(),
34        "data_publicacao": fonte.get("issued") or "",
35        "url": url_final,
36        "pagina_origem": pagina,
37        "coletado_em": coletado_em,
38    }
```

Loop de coleta com *circuit breaker* e deduplicação

coletar_todos()

```
1 def coletar_todos(termo_busca, tamanho_pagina=10, limite_paginas=None):
2     todos, ids_vistos = [], set()
3     from_, pagina = 0, 1
4     falhas, MAX_FALHAS = 0, 3
5     coletado_em = datetime.now().isoformat()
6     total_esperado = float('inf')
7
8     while True:
9         if limite_paginas and pagina > limite_paginas: break
10
11         artigos, total, sucesso = buscar_artigos_com_total(
12             termo_busca, from_, tamanho_pagina)
13         if total > 0: total_esperado = total
14
15         if not artigos:
16             if sucesso: break # busca legit. vazia
17             falhas += 1
18             if falhas >= MAX_FALHAS: break # circuit breaker
19             from_ += tamanho_pagina; pagina += 1
20             if from_ >= total_esperado: break
21             time.sleep(1); continue
22
23         falhas = 0
24         for artigo in artigos:
25             _id = artigo.get("_id")
26             if _id and _id not in ids_vistos:
27                 ids_vistos.add(_id)
28                 todos.append(
29                     extrair_dados_artigo(artigo, pagina, coletado_em))
30
31         from_ += tamanho_pagina; pagina += 1
32         if from_ >= total_esperado: break
33         time.sleep(1)
34
35     return todos
```

Decisões Técnicas

1. **Pular e continuar por página.** Uma falha na página 2 de 100 não deve descartar as 98 restantes. O script registra aviso em log e avança.
2. **Circuit breaker de falhas consecutivas.** Teto de 3 falhas seguidas garante que o loop sempre termina, mesmo que a API fique indisponível desde o início caso em que `total_esperado` permaneceria infinito.
3. **Terceiro valor de retorno (sucesso: bool).** Distingue busca legitimamente vazia (`sucesso=True`, 0 artigos) de falha técnica (`sucesso=False`), evitando logs incorretos e retries desnecessários.
4. **Deduplicação por _id.** Controle em tempo real pelo identificador primário da API, imune a colisões de títulos idênticos.
5. **Timestamp carimbado no nome do arquivo.** Preserva histórico de execuções; atende ao requisito de rastreabilidade.
6. **JSON como formato principal.** Evita corrupção estrutural quando campos como `resumo` contêm vírgulas, aspas ou quebras de linha.
7. **BeautifulSoup mantido para limpeza de HTML.** A migração para a API JSON dispensa o parser de HTML da página, mas a biblioteca é utilizada especificamente para remover tags injetadas no campo `highlight.body`.

Testes Automatizados

Os testes utilizam `unittest` com *fixtures* estáticas que reproduzem variações estruturais reais da API, sem dependência de rede.

`test_extracao.py` cobre o comportamento esperado: decodificação de URL de rastreamento, URL direta sem modificação, prioridade do campo `caption` sobre `summaryBlocks`, *fallbacks* para `summaryBlocks` e `highlight.body` (com limpeza HTML), resumo vazio quando nenhum campo está disponível, consistência das chaves do dicionário de saída e extração correta de datas no formato ISO 8601.

`test_stress.py` verifica robustez: artigo com dicionário vazio (`{}`), campos explicitamente nulos, URL de rastreamento sem o parâmetro `u`, `summaryBlocks` com tipos mistos e blocos corrompidos, e `highlight.body` com tags HTML aninhadas.

Cobertura pendente (não bloqueante): `mocks de requests.post`

para os quatro cenários de resposta de `buscar_artigos_com_total` e testes do `circuit breaker` em `coletar_todos`.

Avaliação de Qualidade dos Dados

Uma amostra de referência com 20 itens foi extraída manualmente via interface web e comparada com os 20 primeiros retornos da rotina automatizada (`from=0, size=20`).

Dimensão	Evidência	Resultado
Compleitude	Título, URL, Data e Página: 20/20. Resumo: 19/20 (1 ausência legítima na base do G1).	100 % / 95 %
Atualidade	Ordenação <code>gl.info_query_recency</code> refletiu 100 % a interface web. Nenhum atraso de indexação.	100 %
Precisão	100 % dos documentos restritos ao domínio G1 via <code>x-tenant-id: gl</code> .	100 %
Acurácia	100 % das URLs de rastreamento decodificadas para a fonte primária.	100 %
Unicidade	Controle por <code>_id</code> . Nenhuma duplicata em 20 registros verificados.	100 %
Consistência	100 % das datas no formato ISO 8601 (<code>issued</code>). Sem variação tipográfica.	100 %
Rastreabilidade	100 % dos registros com <code>coletado_em</code> e arquivo carimbado com <i>timestamp</i> .	100 %

Tabela 1: Dimensões de qualidade dos dados avaliadas sobre amostra de referência ($n = 20$).

Proposta de Suporte via LLM

Contexto e acionamento

A LLM atuará estritamente como agente secundário de manutenção e observabilidade, acionada automaticamente em dois cenários: (1) o *circuit breaker* atinge o limite de falhas consecutivas, ou (2) a estrutura do JSON diverge do schema esperado sinalizado por:

`KeyError` ou `IndexError` em `buscar_artigos_com_total`.

Dados fornecidos ao modelo

O modelo receberá três insumos: o log de erro com *traceback* completo, uma amostra do JSON anômalo capturada em memória no momento da falha e o código atual da função `extrair_dados_artigo`.

Mecanismo de validação (HITL)

A resposta do modelo gera código transitório em `temp_extracao.py`. O sistema executa automaticamente a suíte `TestStressJSON`. Sem aprovação nos testes, o código é descartado. Com aprovação, é salvo em `sugestao_aprovada_llm.py` e encaminhado para revisão humana via *Pull Request* — **Human-in-the-Loop**.

Controles contra alucinação

O prompt inclui a regra: “É proibido supor nomes de campos. Use apenas as chaves explícitas no JSON fornecido.” A temperatura é fixada em 0,1 para máximo determinismo. O código gerado é extraído via *regex* (apenas o bloco “`python`”) e deve passar obrigatoriamente pelos testes antes de qualquer incorporação.

Exemplo de integração

llm_recovery.py

```
1 import os, re, subprocess
2 import google.generativeai as genai
3
4 genai.configure(api_key=os.environ.get("GEMINI_API_KEY"))
5 modelo = genai.GenerativeModel("gemini-2.5-pro")
6
7 def solicitar_correcao_llm(erro_log, json_capturado, codigo_extracao):
8     prompt = (
9         "Voce e um agente de engenharia de dados.\n"
10        f"Erro: {erro_log}\n"
11        f"Codigo atual:\n```\npython\n{codigo_extracao}\n```\n"
12        f"JSON da API:\n```\njson\n{json_capturado}\n```\n"
13        "Regra: use APENAS as chaves do JSON. "
14        "Retorne somente codigo Python."
15    )
16    r = modelo.generate_content(
17        prompt, generation_config={"temperature": 0.1})
18    return re.search(
19        r"```python\n(.*?)```", r.text, re.DOTALL).group(1)
20
21 def validar_e_encaminhar(codigo_gerado):
22     with open("temp_extracao.py", "w") as f: f.write(codigo_gerado)
23     resultado = subprocess.run(
24         ["python", "-m", "unittest", "tests/test_stress.py"],
25         capture_output=True, text=True
26     )
27     os.remove("temp_extracao.py")
28     if resultado.returncode == 0:
29         with open("sugestao_aprovada_llm.py", "w") as f:
30             f.write(codigo_gerado)
31         print("Aprovado. Aguardando revisao humana (PR).")
32     else:
33         print("Rejeitado. Intervencao manual necessaria.")
```

Limitações

Limitações atuais

A principal limitação é a dependência de um contrato de API não documentado publicamente: mudanças no schema de `busca.globo.com/v1/search` podem quebrar a coleta sem aviso. A cobertura de testes é parcial, `buscar_artigos_com_total` e o *circuit breaker* em `coletar_todos` ainda carecem de mocks de rede. A estratégia de espera entre requisições usa 1 segundo fixo, sem *backoff* exponencial. A amostra de referência de 20 itens é suficiente para prova de conceito, mas insuficiente para inferência estatística robusta. Por fim, a integração com LLM está documentada com exemplo funcional, mas não foi integrada ao pipeline de produção como solicitado no case.

Repositório Git: https://github.com/hellnM/2etapa_netlab-ufrj.git — inclui código-fonte, base coletada, testes executados e métricas de qualidade.